```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
import plotly.io as pio

pio.templates.default = "plotly_white"
# If charts do not display (for example in VS Code or Colab), uncomment one line:
# pio.renderers.default = "notebook_connected"
# pio.renderers.default = "colab"

print("Plotly version:", __import__("plotly").__version__)
```

```
Plotly version: 5.24.1
```

```
df = pd.read_csv("earthquake_1995-2023.csv")
print("Shape:", df.shape)
df.head()
```

```
Shape: (1000, 19)
```

	title	magnitude	date_time	cdi	mmi	alert	tsunami	sig	net	nst	dmin	gap	magType	depth	latitude	longitude
0	M 6.5 - 42 km W of Sola, Vanuatu	6.5	16/08/2023 12:47	7	4	green	0	657	us	114	7.177000	25.0	mww	192.955	-13.8814	167.1580
1	M 6.5 - 43 km S of Intipucá, El Salvador	6.5	19/07/2023 00:22	8	6	yellow	0	775	us	92	0.679000	40.0	mww	69.727	12.8140	-88.1265
2	M 6.6 - 25 km ESE of Loncopué, Argentina	6.6	17/07/2023 03:05	7	5	green	0	899	us	70	1.634000	28.0	mww	171.371	-38.1911	-70.3731
	M 7.2 - 98															

Next steps: Generate code with df | New interactive sheet

```
# Data types and missing values per column
summary = pd.DataFrame({"dtype": df.dtypes.astype(str),
                        "missing": df.isna().sum(),
                        "missing_%": (df.isna().mean() * 100).round(1)})
summary
```

	dtype	missing	missing_%
title	object	0	0.0
magnitude	float64	0	0.0
date_time	object	0	0.0
cdi	int64	0	0.0
mmi	int64	0	0.0
alert	object	551	55.1
tsunami	int64	0	0.0
sig	int64	0	0.0
net	object	0	0.0
nst	int64	0	0.0
dmin	float64	0	0.0
gap	float64	0	0.0
magType	object	0	0.0
depth	float64	0	0.0
latitude	float64	0	0.0
longitude	float64	0	0.0
location	object	6	0.6
continent	object	716	71.6
country	object	349	34.9

Next steps: Generate code with summary | New interactive sheet

```
df[["magnitude", "depth", "sig", "mmi", "cdi", "tsunami"]].describe().round(2)
```

	magnitude	depth	sig	mmi	cdi	tsunami
count	1000.00	1000.00	1000.00	1000.00	1000.00	1000.00
mean	6.94	74.61	847.92	6.03	3.60	0.32
std	0.44	130.81	301.80	1.43	3.33	0.47
min	6.50	2.70	650.00	1.00	0.00	0.00
25%	6.60	16.00	691.00	5.00	0.00	0.00
50%	6.80	29.00	744.00	6.00	4.00	0.00
75%	7.10	55.00	874.25	7.00	7.00	1.00
max	9.10	670.81	2910.00	10.00	9.00	1.00

```
df["date_time"] = pd.to_datetime(df["date_time"], format="%d/%m/%Y %H:%M")
df["year"] = df["date_time"].dt.year

df["alert"] = df["alert"].fillna("none")
df["location"] = df["location"].fillna("Unknown")
df["country"] = df["country"].fillna("Unknown")

# Fixed category order and colours, reused in every chart for consistency
ALERT_ORDER = ["green", "yellow", "orange", "red", "none"]
ALERT_COLORS = {"green": "#2ca02c", "yellow": "#e6b800", "orange": "#ff7f0e",
                "red": "#d62728", "none": "#9e9e9e"}

DEPTH_TICKS = [3, 10, 30, 100, 300, 600]   # clean tick labels for log-scaled depth axes

df = df.sort_values("date_time").reset_index(drop=True)
print(df["alert"].value_counts().reindex(ALERT_ORDER))
print("Date range:", df["date_time"].min().date(), "to", df["date_time"].max().date())
```

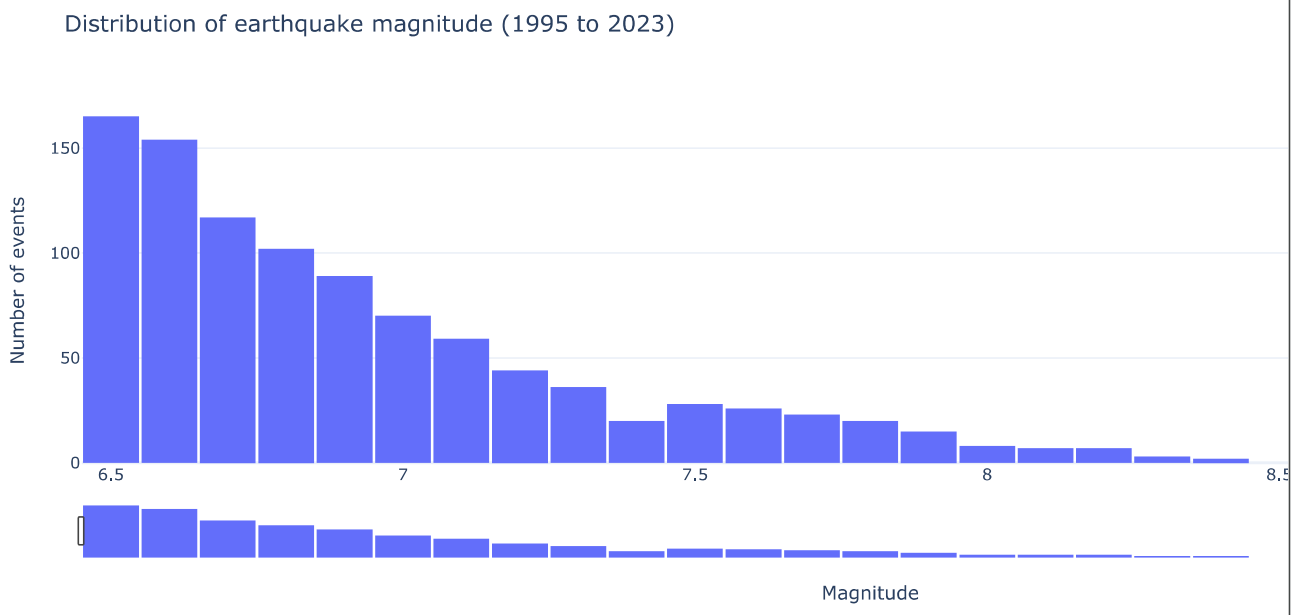
```
alert
green     353
yellow     59
```

```
orange     24
red        13
none      551
Name: count, dtype: int64
Date range: 1995-05-05 to 2023-08-16
```

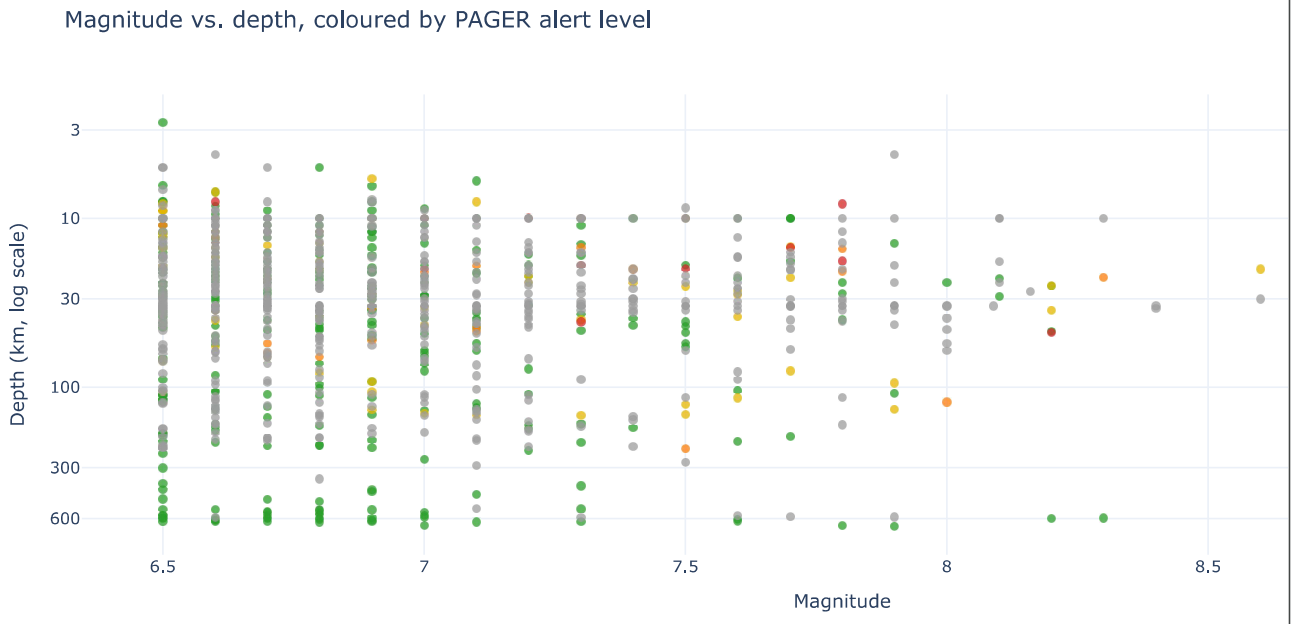
```
# Quick check: in which years is the alert level actually recorded?
(df.assign(recorded=df["alert"] != "none")
   .groupby("year")["recorded"].mean().mul(100).round(0)
   .rename("% events with alert").to_frame().T)
```

year	1995	1996	1997	1998	1999	2000	2001	2002	2003	2004	...	2014	2015	2016	2017	2018	2019	2020	2021	2022	2
% events with	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	100.0	100.0	100.0	97.0	100.0	97.0	100.0	100.0	90.0	

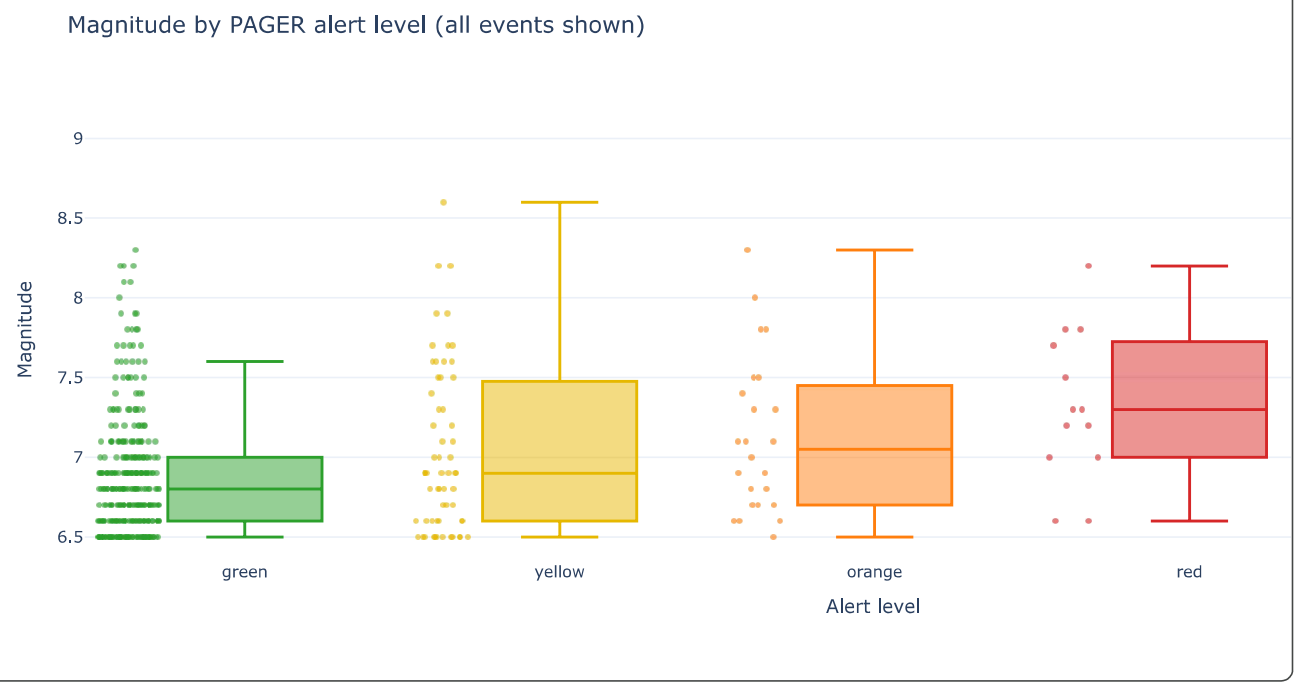
```
fig1 = px.histogram(df, x="magnitude",
                    title="Distribution of earthquake magnitude (1995 to 2023)",
                    labels={"magnitude": "Magnitude"})
# Magnitudes are reported to 0.1, so use bins exactly 0.1 wide and centred on each value.
# (Automatic bins of width 0.2 merge 6.6 and 6.7 into one bar and create a false peak.)
fig1.update_traces(xbins=dict(start=6.45, end=9.15, size=0.1))
fig1.update_layout(yaxis_title="Number of events", bargap=0.05,
                   xaxis=dict(rangeslider=dict(visible=True, thickness=0.12), dtick=0.5))
# Note: with a range slider, Plotly always draws the x-axis title BELOW the slider.
fig1.show()
```




```
fig2 = px.scatter(df, x="magnitude", y="depth", color="alert",
                  category_orders={"alert": ALERT_ORDER},
                  color_discrete_map=ALERT_COLORS,
                  hover_name="title",
                  hover_data={"location": True, "country": True,
                              "date_time": "|%d %b %Y %H:%M", "sig": True,
                              "alert": False},
                  log_y=True, opacity=0.75,
                  labels={"magnitude": "Magnitude", "depth": "Depth (km, log scale)",
                          "alert": "Alert level"},
                  title="Magnitude vs. depth, coloured by PAGER alert level")
fig2.update_yaxes(autorange="reversed", tickvals=DEPTH_TICKS)   # deeper events plotted lower, like a cross-section
fig2.show()
```




```
fig3 = px.box(df, x="alert", y="magnitude", color="alert",
              category_orders={"alert": ALERT_ORDER},
              color_discrete_map=ALERT_COLORS,
              points="all", hover_name="title",
              hover_data={"date_time": "|%d %b %Y", "depth": True, "alert": False},
              labels={"alert": "Alert level", "magnitude": "Magnitude"},
              title="Magnitude by PAGER alert level (all events shown)")
fig3.update_traces(marker=dict(size=4, opacity=0.6), jitter=0.4)
fig3.update_layout(showlegend=False)
fig3.show()
```

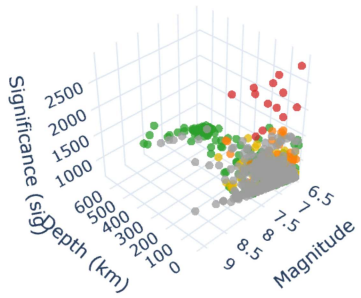



```
# Numbers behind the box plot
df.groupby("alert")[["magnitude", "sig"]].median().reindex(ALERT_ORDER).round(2)
```

alert	magnitude	sig
green	6.80	733.0
yellow	6.90	972.0
orange	7.05	1429.5
red	7.30	2504.0
none	6.80	737.0

```
fig4 = px.scatter_3d(df, x="magnitude", y="depth", z="sig", color="alert",
                     category_orders={"alert": ALERT_ORDER},
                     color_discrete_map=ALERT_COLORS,
                     hover_name="title", opacity=0.75,
                     labels={"magnitude": "Magnitude", "depth": "Depth (km)",
                             "sig": "Significance (sig)", "alert": "Alert level"},
                     title="Magnitude, depth and significance in 3D (rotate and zoom)")
fig4.update_traces(marker=dict(size=3.5))
fig4.update_layout(height=650, scene=dict(yaxis=dict(autorange="reversed")))
fig4.show()
```

Magnitude, depth and significance in 3D (rotate and zoom)

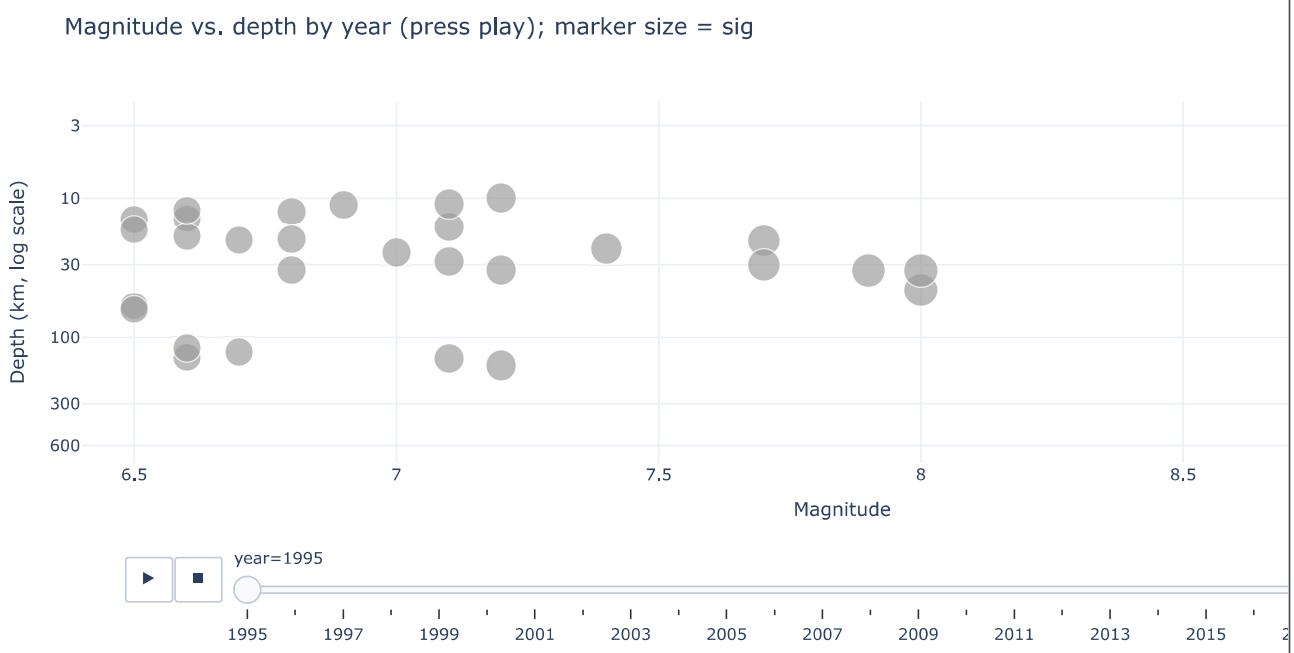



```
years = sorted(df["year"].unique())
full_grid = pd.MultiIndex.from_product([years, ALERT_ORDER], names=["year", "alert"]).to_frame(index=False)
present = df[["year", "alert"]].drop_duplicates()
missing = full_grid.merge(present, how="left", indicator=True).query("_merge == 'left_only'")[["year", "alert"]]

anim_df = pd.concat([df, missing.assign(magnitude=np.nan, depth=np.nan, sig=0,
                                        title="", location="", country="")],
                    ignore_index=True).sort_values(["year", "alert"])

fig5 = px.scatter(anim_df, x="magnitude", y="depth", color="alert", size="sig",
                  animation_frame="year",
                  category_orders={"alert": ALERT_ORDER, "year": years},
                  color_discrete_map=ALERT_COLORS,
                  hover_name="title", hover_data={"location": True, "sig": True, "year": False},
                  log_y=True, size_max=30,
                  range_x=[6.4, 9.3], range_y=[800, 2],   # deep at the bottom; px applies log10 itself
```

```
                  labels={"magnitude": "Magnitude", "depth": "Depth (km, log scale)",
                          "alert": "Alert level"},
                  title="Magnitude vs. depth by year (press play); marker size = sig")
fig5.update_yaxes(tickvals=DEPTH_TICKS)
fig5.update_layout(legend_itemsizing="constant")   # legend dots stay visible even when sig is small
fig5.layout.updatemenus[0].buttons[0].args[1]["frame"]["duration"] = 700
fig5.show()
```




```
# Events per year, used in the comparison below
df.groupby("year").size().rename("events").to_frame().T
```

year	1995	1996	1997	1998	1999	2000	2001	2002	2003	2004	...	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023
events	29	35	28	23	33	35	28	25	31	32	...	48	53	43	36	43	33	27	42	41	34

1 rows × 29 columns

```
metrics = {
    "depth": ("Depth (km, log scale)", "log", "array"),
    "sig":   ("Significance (sig)", "linear", "auto"),
    "mmi":   ("Max. instrumental intensity (MMI)", "linear", "auto"),
    "cdi":   ("Max. reported intensity (CDI)", "linear", "auto"),
}

colors = df["alert"].map(ALERT_COLORS)
fig6 = go.Figure(go.Scatter(
    x=df["magnitude"], y=df["depth"], mode="markers",
    marker=dict(color=colors, size=7, opacity=0.7, line=dict(width=0.5, color="white")),
    text=df["title"], customdata=df["alert"],
    hovertemplate="<b>%{text}</b><br>Magnitude: %{x}<br>Value: %{y}<br>Alert: %{customdata}<extra></extra>",
))

buttons = []
for col, (label, axis_type, tickmode) in metrics.items():
    buttons.append(dict(
        label=col,
        method="update",
        args=[{"y": [df[col]]},                                  # data update
              {"yaxis.title.text": label, "yaxis.type": axis_type,  # layout update
               "yaxis.tickmode": tickmode, "yaxis.tickvals": DEPTH_TICKS if col == "depth" else None,
               "yaxis.autorange": "reversed" if col == "depth" else True,
               "title.text": f"Magnitude vs. {col}"}],
    ))

fig6.update_layout(
    title="Magnitude vs. depth",
    xaxis_title="Magnitude", yaxis_title=metrics["depth"][0], yaxis_type="log", yaxis_tickvals=DEPTH_TICKS, yaxis_autorange="r
```

```
    updatemenus=[dict(buttons=buttons, direction="down", x=1.0, xanchor="right",
                      y=1.15, yanchor="top", showactive=True)],
    annotations=[dict(text="y-axis:", x=0.915, xref="paper", y=1.12, yref="paper",
                      showarrow=False, xanchor="right")],
)
```




```
# Spearman correlations: what the dropdown lets you compare
df[["magnitude", "depth", "sig", "mmi", "cdi"]].corr(method="spearman").loc["magnitude"].round(2)
```

	magnitude
magnitude	1.00
depth	0.10
sig	0.79